

# **CompE565, Spring 2026 Project**

**Prepared by**  
*Rhilo Sotto*  
130551574

Email: [rsotto4586@sdsu.edu](mailto:rsotto4586@sdsu.edu)  
Electrical and Computer Engineering  
San Diego State University

## Table of Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Table of Contents.....</b>                                  | <b>2</b>  |
| <b>Introduction.....</b>                                       | <b>2</b>  |
| <b>Procedure.....</b>                                          | <b>3</b>  |
| <b>Results.....</b>                                            | <b>3</b>  |
| <b>List of Figures.....</b>                                    | <b>4</b>  |
| Figure 1: Original 16th Frame.....                             | 4         |
| Figure 2: Original 17th Frame.....                             | 4         |
| Figure 3: Original 18th Frame.....                             | 5         |
| Figure 4: Original 19th Frame.....                             | 5         |
| Figure 5: Original 20th Frame.....                             | 6         |
| Figure 6: Predicted 17th Frame.....                            | 6         |
| Figure 7: Predicted 18th Frame.....                            | 7         |
| Figure 8: Predicted 19th Frame.....                            | 7         |
| Figure 9: Predicted 20th Frame.....                            | 8         |
| Figure 10: 17th Difference Frame.....                          | 8         |
| Figure 11: 18th Difference Frame.....                          | 9         |
| Figure 12: 19th Difference Frame.....                          | 9         |
| Figure 13: 20th Difference Frame.....                          | 10        |
| Figure 14: Reconstructed 16th Frame.....                       | 10        |
| Figure 15: Reconstructed 17th Frame.....                       | 11        |
| Figure 16: Reconstructed 18th Frame.....                       | 11        |
| Figure 17: Reconstructed 19th Frame.....                       | 12        |
| Figure 18: Reconstructed 20th Frame.....                       | 12        |
| Figure 19: PSNR (Y-component) of each Reconstructed Frame..... | 13        |
| <b>Conclusion.....</b>                                         | <b>13</b> |

## Introduction

The purpose of this project was to implement a simple MPEG-like video encoder and decoder in MATLAB. The project combines the ideas from the previous assignments, including color conversion, 4:2:0 chroma subsampling, block DCT, quantization, zigzag scanning, DC differential coding, motion estimation, and motion compensation.

The codec was tested using five frames from the foreman\_cif.avi video sequence. The first frame was encoded as an I-frame, and the next four frames were encoded as P-frames. The implementation does not include entropy coding, multiplexing, or buffer control, as these were not required for the project.

This project helped me see how video compression reduces both spatial redundancy within a frame and temporal redundancy between frames. Spatial redundancy is reduced using the DCT and quantization, while temporal redundancy is reduced using motion estimation and motion-compensated prediction.

## Procedure

I used MATLAB's **VideoReader** to load `foreman_cif.avi` and selected frames 16–20 for processing. Each frame was converted from RGB to YCbCr using a custom BT.601-style helper function, and the Cb and Cr planes were reduced using **subsample420()**. Since the first selected frame was used as the I-frame, I encoded it independently using 8x8 DCT blocks, quantization with  $Q = 28$ , zigzag scanning, DC differential coding, and coefficient storage. Helper functions such as **encodePlane()**, **dctMatrix()**, **zigzagScan()**, **inverseZigzag()**, and **decodePlane()** were used for these block operations.

Frames 2 through 5 were encoded as P-frames. Motion estimation was performed on  $16 \times 16$  Y macroblocks using **motionEstimateFullSearch()** with a  $\pm 8$  pixel search range and sum of absolute differences, or SAD, as the matching criterion. Predicted frames were generated using **motionCompensateY()** and **motionCompensateChroma()**. After generating the prediction, I subtracted it from the original frame to get the residual. I then encoded that residual using the same DCT, quantization, zigzag scanning, and DC differential coding steps.

The decoder was implemented using **decodeVideo()**. For the I-frame, the decoder reconstructed the frame from the stored coefficients using inverse zigzag, inverse quantization, and inverse DCT. For each P-frame, the decoder used the stored motion vectors to regenerate the prediction, decoded the residual, and added the residual back to the predicted frame. The reconstructed frames were converted back to RGB using **ycbcr420ToRGB()**, and PSNR was calculated on the Y component using **computePSNR()**. I kept the decoding code separate from the encoding code and implemented separate inverse functions for reconstruction.

## Results

The MATLAB program displayed and saved the original five frames, predicted P-frames, reconstructed difference P-frames, and reconstructed frames. The predicted and difference frames were shown only for frames 2 through 5 because frame 1 was an I-frame and did not use prediction. The generated frames and PSNR result are shown below:

## List of Figures



*Figure 1: Original 16th Frame*



*Figure 2: Original 17th Frame*



*Figure 3: Original 18th Frame*



*Figure 4: Original 19th Frame*



*Figure 5: Original 20th Frame*



*Figure 6: Predicted 17th Frame*



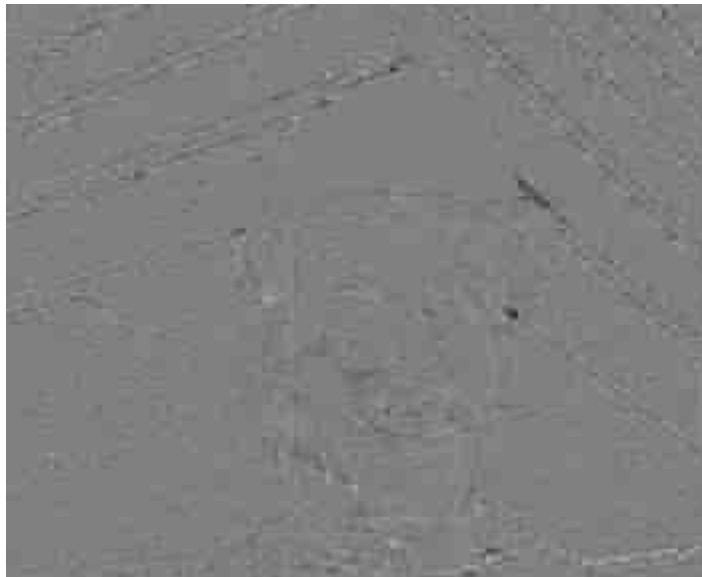
*Figure 7: Predicted 18th Frame*



*Figure 8: Predicted 19th Frame*



*Figure 9: Predicted 20th Frame*



*Figure 10: 17th Difference Frame*





*Figure 11: 18th Difference Frame*



*Figure 12: 19th Difference Frame*



*Figure 13: 20th Difference Frame*



*Figure 14: Reconstructed 16th Frame*



*Figure 15: Reconstructed 17th Frame*



*Figure 16: Reconstructed 18th Frame*



*Figure 17: Reconstructed 19th Frame*



*Figure 18: Reconstructed 20th Frame*

| PSNR Results for Y Component |      |           |
|------------------------------|------|-----------|
| Frame                        | Type | PSNR-Y dB |
| 16                           | I    | 29.3278   |
| 17                           | P    | 29.6301   |
| 18                           | P    | 29.5311   |
| 19                           | P    | 29.4047   |
| 20                           | P    | 29.4264   |

*Figure 19: PSNR (Y-component) of each Reconstructed Frame*

The reconstructed frames looked close to the original frames overall, but some blur and loss of detail were visible around the face and edges. This loss was mainly caused by quantization and by keeping only the DC coefficient plus the first seven AC coefficients from each 8x8 DCT block. The difference frames made it easier to see where motion compensation worked well and where it struggled. The largest residuals appeared around the face edges and facial features, where the motion was harder to predict accurately.

## Conclusion

In this project, a simplified MPEG-like codec was implemented in MATLAB. The codec used one I-frame followed by four P-frames. The video was converted to YCbCr format, subsampled using 4:2:0 sampling, and coded using DCT, quantization, zigzag scanning, DC differential coding, motion estimation, and motion compensation. The decoder reconstructed the frames using only the coded information from the encoder, including frame type, quantized coefficients, motion vectors, and the quantizer value. The original frames were only used after decoding to calculate PSNR.

The main tradeoff I noticed was between compression and visual quality. Since I only kept the DC coefficient and the first seven AC coefficients from each 8x8 block, the reconstructed frames still followed the original motion, but fine details around the face were blurred. Increasing the number of stored coefficients would likely improve the PSNR, but it would also increase the amount of data that needs to be stored.

Overall, this project helped me understand how the main parts of an MPEG-style codec connect together, from prediction and residual coding to reconstruction and PSNR measurement.